

Simplicio Loop — Loop stack economy benchmark

Why the numbers look the way they do · multi-issue matrix · Fast is part of loop

```
Stack measured: {"simplicio-loop": "simplicio-loop 3.38.10", "simplicio-mapper": "0.26.0", "simplicio-fast": "simplicio-fast 2.0.0"}
Issues (old→new): [9, 96, 171, 322, 711]
Lanes: ['baseline', 'loop', 'loop_mcp', 'mcp_only', 'loop_no_fast']
Token method: context_bytes / 4 (rough; not provider-billed; no double-count of same dump)
Runs: 25 | schema: simplicio.bench.multi-issue-lanes/v1
```

TL;DR

- STRICT loop already includes Fast when operational (mapper · fast · dev-cli).
- “% savings” is vs a host full-file baseline – not a quality score.
- loop_no_fast often “saves more tokens” because it does LESS work (no Fast packets).
- mcp_only ~99% savings = almost no repo survey (tool metadata), not better delivery.
- loop_mcp ≈ loop on tokens (+small overhead) – MCP is a capability bus, not the main compressor.
- Wall clock is high on cold monorepo operator startup; multi-turn cost tracks tokens more.

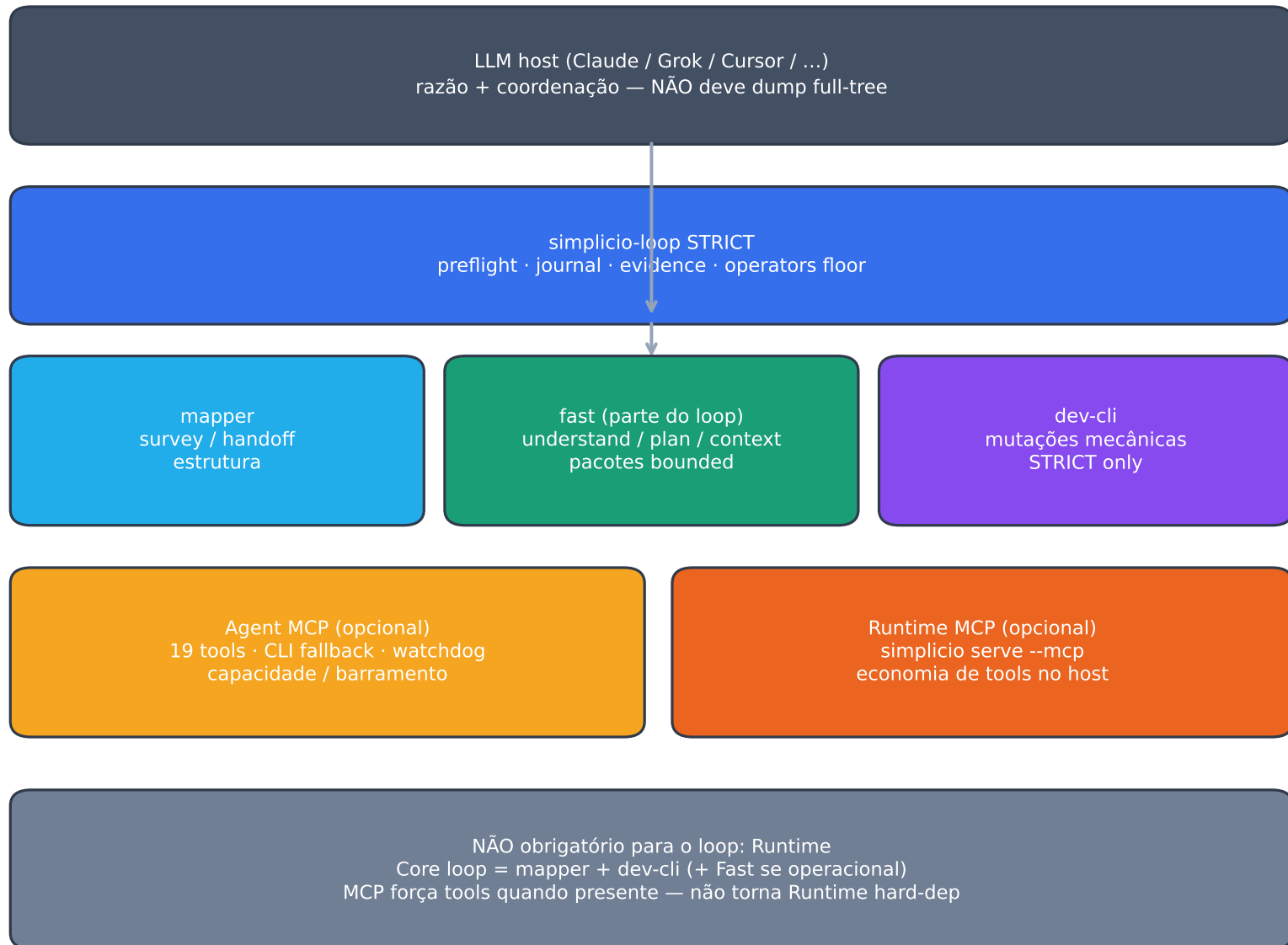
Mean token savings vs baseline (5 issues):

loop	mean	38.7%	med	33.9%	[-31.4 .. 83.1]
loop_mcp	mean	38.5%	med	33.6%	[-31.9 .. 83.0]
mcp_only	mean	99.2%	med	99.1%	[98.2 .. 99.8]
loop_no_fast	mean	63.7%	med	60.9%	[22.1 .. 90.0]

Mean wall / tokens / fast_calls:

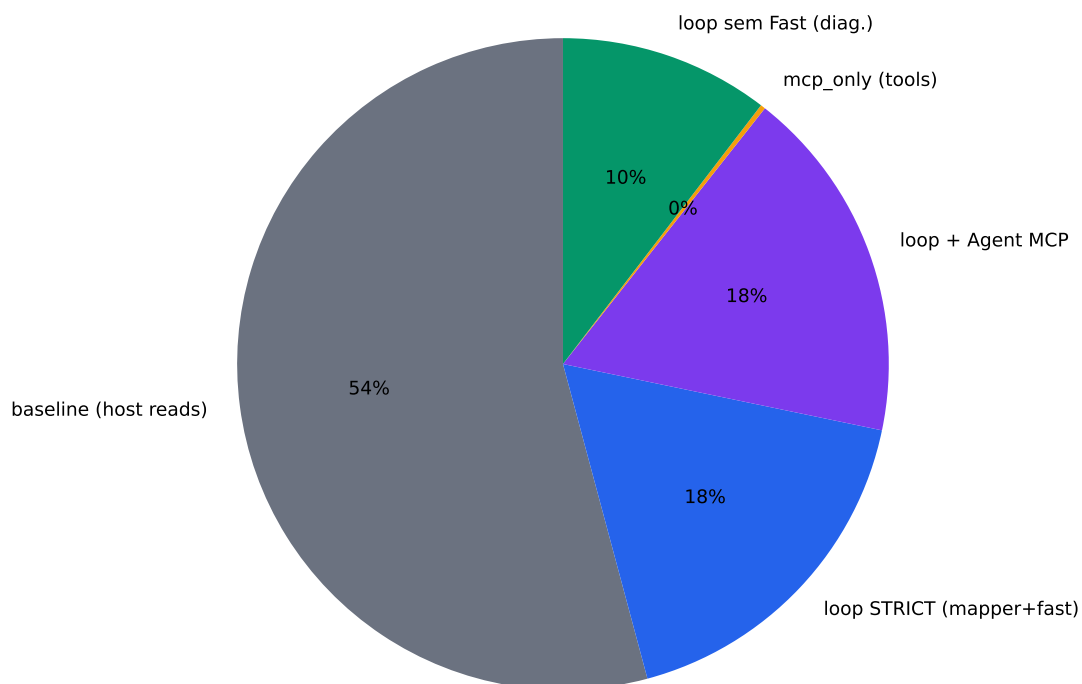
baseline	wall	0.2s	tok	65246	fast	0.0	success	100%
loop	wall	49.7s	tok	21149	fast	5.0	success	100%
loop_mcp	wall	53.2s	tok	21228	fast	5.0	success	100%
mcp_only	wall	11.2s	tok	290	fast	0.0	success	100%
loop_no_fast	wall	17.8s	tok	12522	fast	0.0	success	100%

Barramento do stack (contrato STRICT) — o que cada camada faz

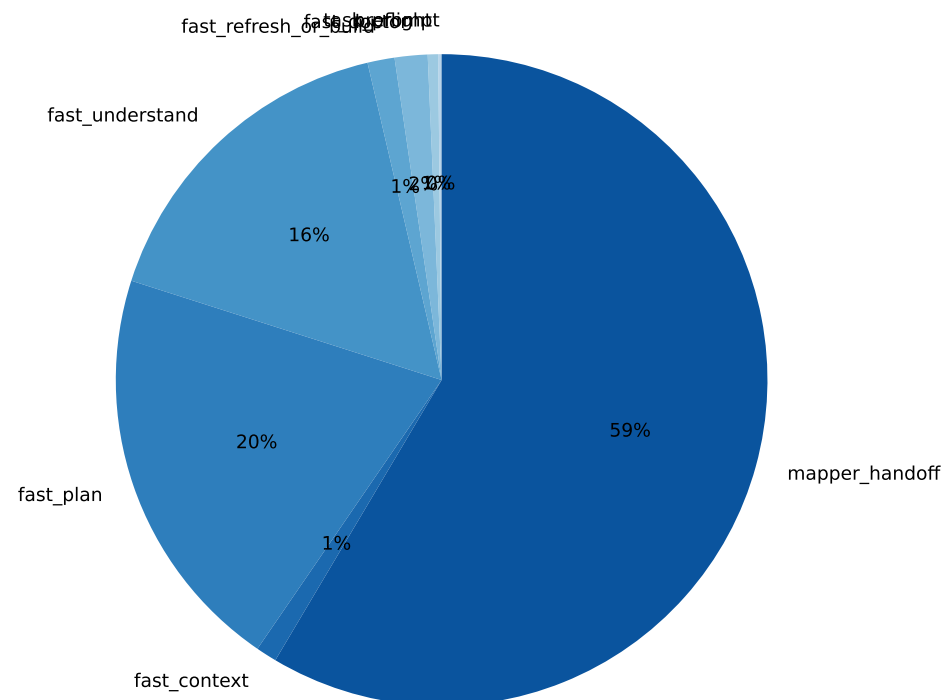


Pizza — composição de contexto

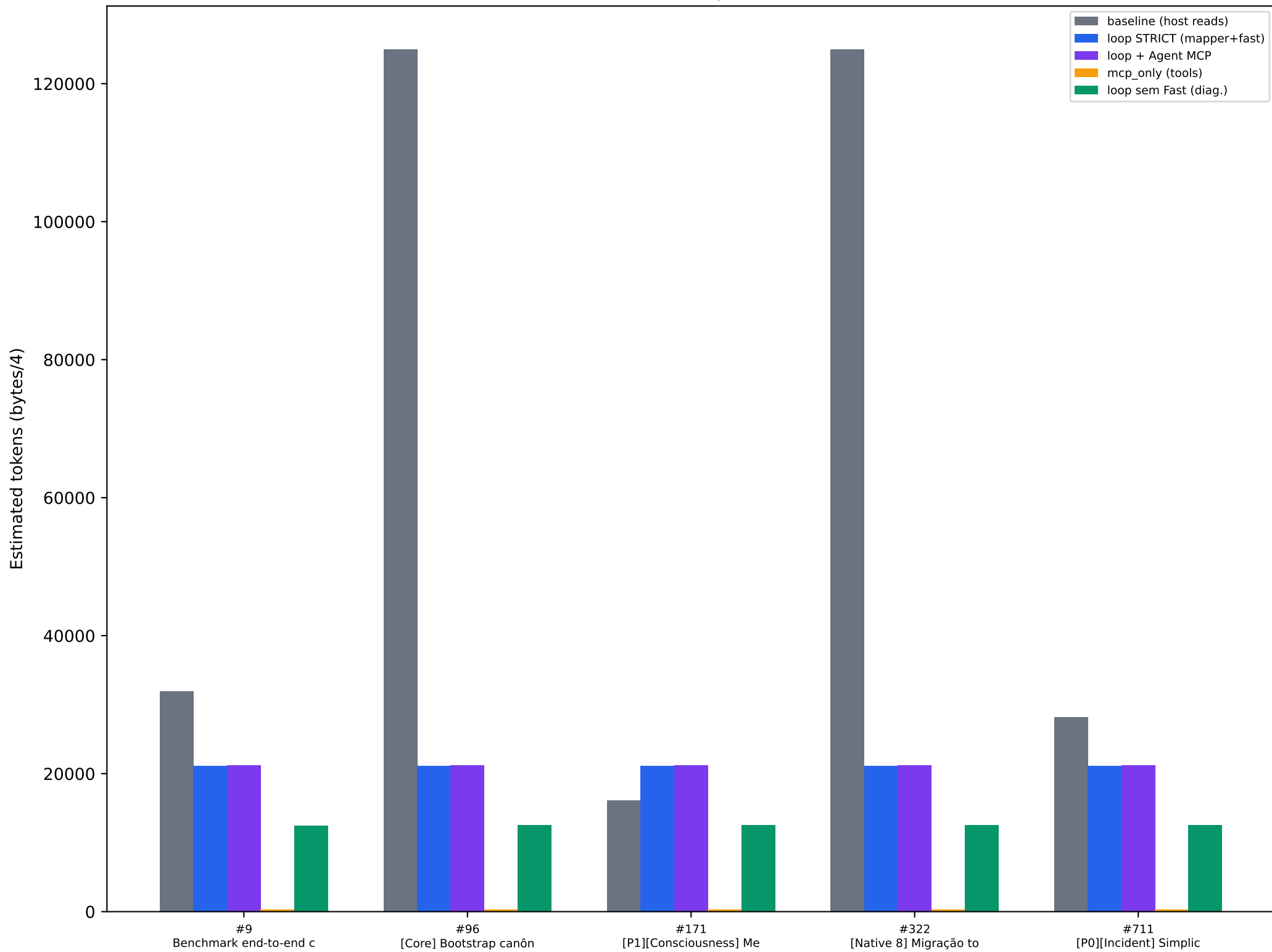
Share of mean est. tokens by lane
(relative size of intake)



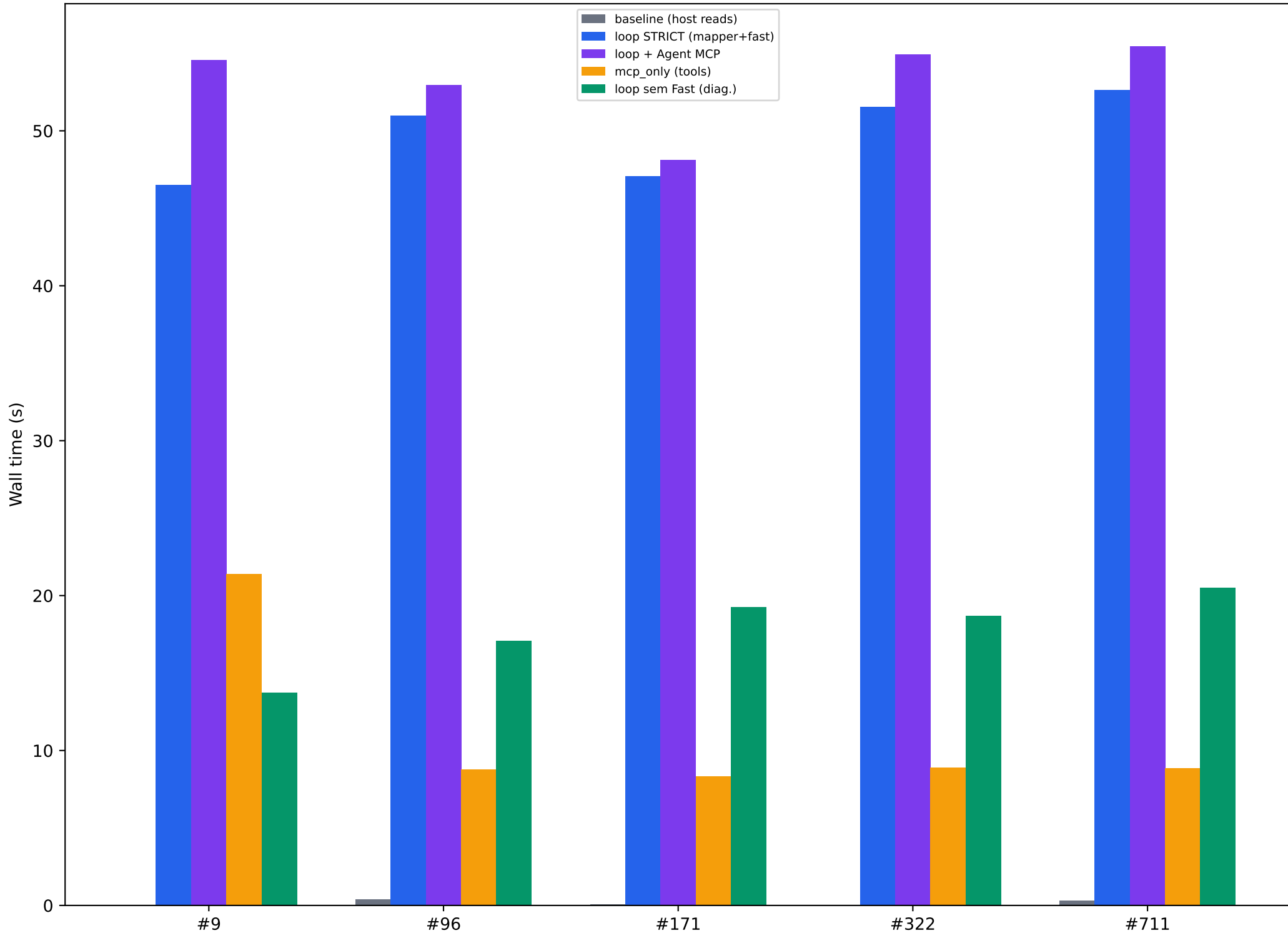
Where loop tokens go (mean across issues)
Fast packets dominate on purpose



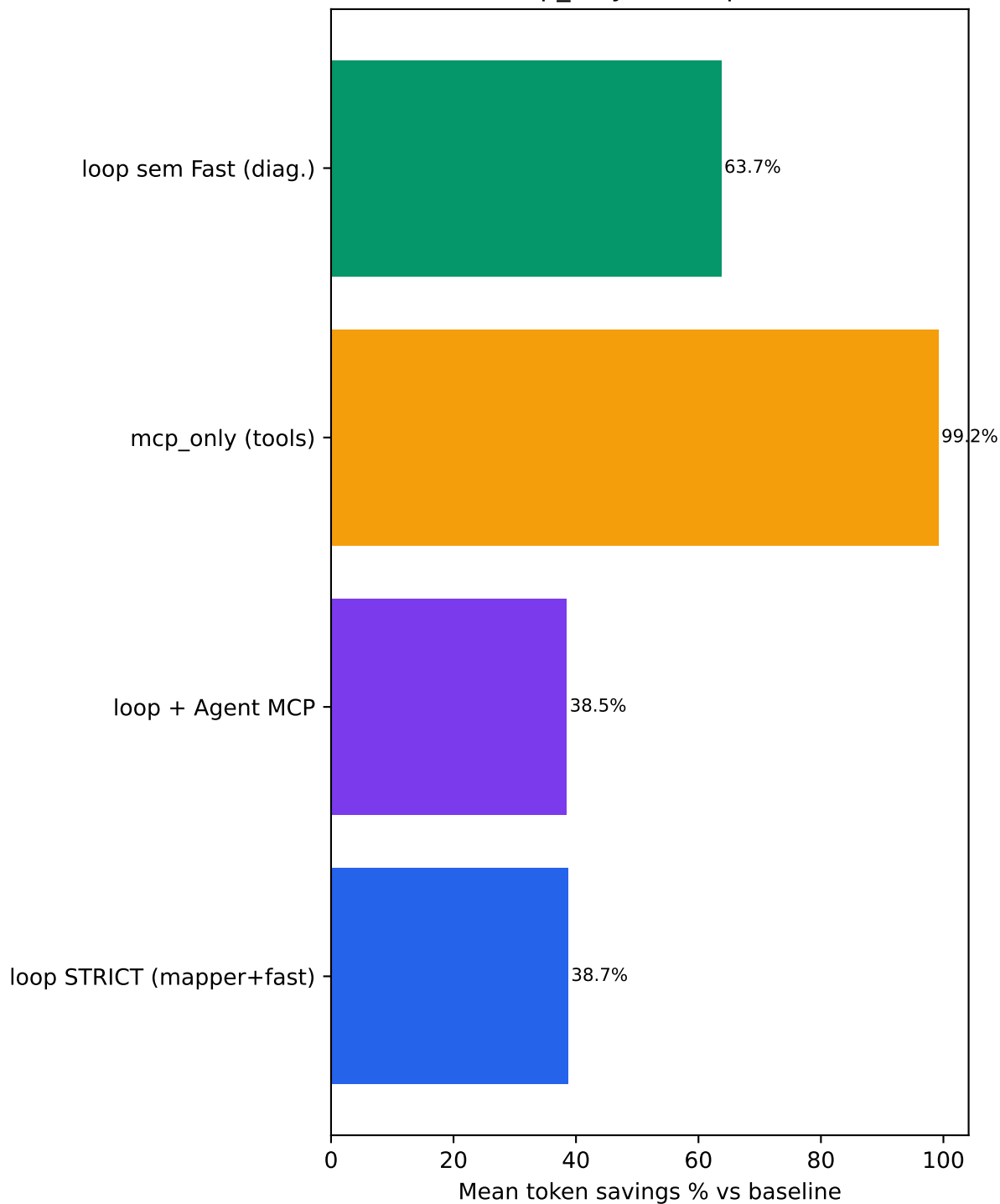
Barras — est. tokens por issue × lane



Barras — wall clock frio por issue × lane
(operators perdem no cold start; baseline é leitura local)



Economia média (5 issues)
Cuidado: mcp_only alto = pouco trabalho



Fast está atuando?
(mean fast_calls / issue)

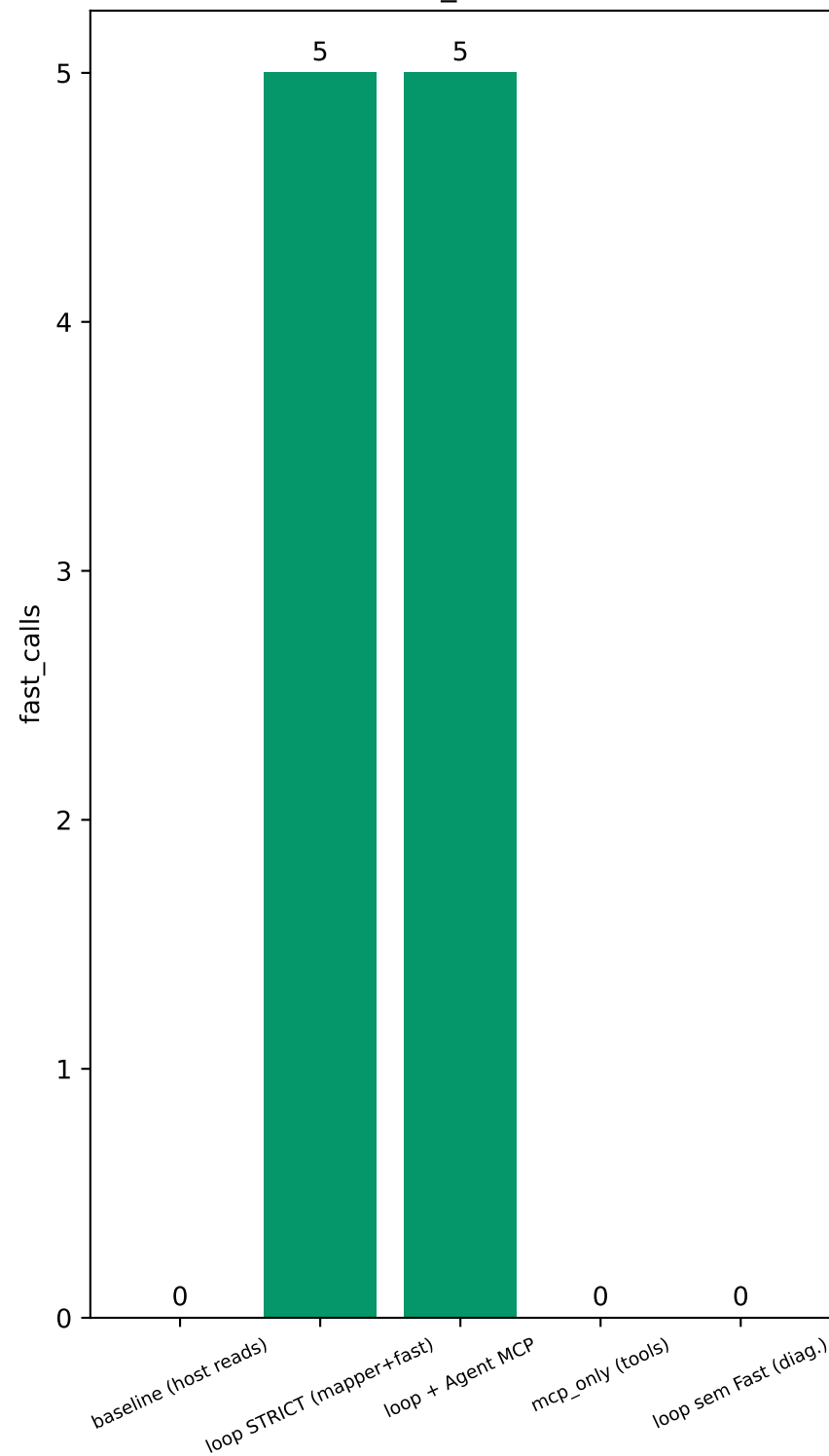


Tabela — savings % vs baseline por issue
Negativo (#171) = baseline pequeno < piso semântico do loop (~21k tok)

Issue	baseline tok	loop %	loop_mcp %	loop_no_fast %	mcp_only %
#9	31965	33.89	33.64	60.87	99.1
#96	125002	83.09	83.03	89.99	99.77
#171	16109	-31.44	-31.92	22.12	98.19
#322	125003	83.08	83.02	89.98	99.77
#711	28150	24.87	24.59	55.52	98.95

Interpretação detalhada

POR QUE OS RESULTADOS SAÍRAM ASSIM

- 1) Três eixos misturados
 - Economia de contexto (tokens estimados = context_bytes/4)
 - Trabalho real (orientar / planejar / tools)
 - Wall-clock frio (subir preflight/mapper/fast no monorepo)Ler só “% savings” como “melhor sistema” é erro.
- 2) Baseline ≠ tratamento

Host full-file: barato em CPU, caro em tokens se a árvore for grande.
Loop STRICT: mais wall frio, contexto empacotado (mapper + Fast).
loop_no_fast: menos texto que full loop porque OMITTE Fast.
mcp_only: quase zero survey de repo → % altíssimo e enganoso.
- 3) Fast É parte do loop STRICT

Contrato: mapper · fast (se operacional) · dev-cli.
Nos runs loop/loop_mcp: fast_calls = 5 sempre (doctor, refresh/build, understand, plan, context). Não é “ligar Fast de novo”.
- 4) Onde vão os tokens do loop

A maior parte vem de understand/plan/context do Fast (~15–18k),
depois mapper handoff. Isso é piso semântico útil, não desperdício.
- 5) MCP quase não corta tokens

loop_mcp ≈ loop (+~80 tok, +~3–4 s). MCP = barramento/tools/governança,
não compressor principal. Runtime MCP no host é outra superfície.
- 6) Variância entre issues

Baseline grande (#96/#322 ~125k) → loop ~83% savings.
Baseline médio (#9/#711) → ~25–34%.
Baseline pequeno (#171 ~16k) → loop “pior” em tokens (-31%) e ainda assim
é o path STRICT correto (mais orientação).
- 7) Wall ~50s no loop

Cold start monorepo (preflight + Fast + mapper). Multi-turn billed cost
segue tokens, não esses segundos frios.

CONCLUSÃO DE PRODUTO

- Manter loop STRICT + Fast quando Fast up.
- Não vender MCP como economizador de tokens; vender como bus.
- Reportar sempre: tokens + wall frio + sucesso/trabalho – nunca só %.
- loop_no_fast e mcp_only são diagnósticos, não vitórias de economia.

Next steps

0 QUE EU FARIA A SEGUIR (recomendação)

1. Scorecard único no README
tokens_saved · wall_cold · work_done · success
(impede mcp_only 99% parecer vitória)
2. Bench “issue done”
Mesma issue → PR + testes nas lanes, com snapshot Fast quente e tokens de sessão LLM reais (provider-billed), não só bytes/4
3. Separar superfícies MCP no marketing
Agent MCP serve vs Runtime MCP (simplicio serve --mcp)
Ambos “MCP”, papéis diferentes
4. Warm path documentado
Publicar números frios E quentes (2ª+ iteração) lado a lado
5. Baseline policy
Mesmo conjunto de arquivos “related” versionado no fixture para issues com baseline artificialmente pequeno
6. Telemetria contínua
Journal loop: operator_calls, context_bytes, fast_calls por run
Dashboard simples no loop/runtime doctor

REPRODUZIR

```
# no simplicio-agent (harness completo ~15-20 min)
python scripts/bench_multi_issue_lanes.py
```

```
# gerar este PDF a partir do JSON
python scripts/render_loop_stack_economy_report_pdf.py \
  --metrics docs/evidence/multi_issue_lanes_metrics.json \
  --out docs/evidence/loop_stack_economy_benchmark_report.pdf
```

Artefatos relacionados

- multi_issue_lanes_metrics.json – raw runs + aggregate
- multi_issue_lanes_bench.pdf – charts do harness multi-issue
- issue27_loop_fast_mcp_* – slice MCP bus #27
- Este arquivo – interpretação + pizza + barras + barramento

Repos

```
{'9': {'number': 9, 'title': 'Benchmark end-to-end comparando Simplicio vs Hermes upstream', 'state': 'CLOSED', 'createdAt': '2026-07-02T05
```